Лабораторная работа 6. Выбор и обучение нейронной сети по размеченным данным

Цель работы

Приобрести навыки выбора нейронной сети для обнаружения производственных объектов, подготовки параметров обучения и оценки полученной модели по валидационной выборке. Выполнить два сопоставимых опыта на наборе данных лабораторной работы №5 и сохранить модель для проверки на видеокадрах в лабораторной работе №7.

Теоретические сведения

В лабораторной работе №3 определена последовательность предварительной обработки изображений, в работе №4 исследованы признаки производственного объекта, а в работе №5 сформирован размеченный набор данных. На следующем этапе изображения и соответствующие им аннотации используются для обучения нейронной сети. Результатом работы является файл весов модели, перечень классов и описание условий, при которых выполнено обучение.

Во всех вариантах решается задача обнаружения объектов с определением их классов и ограничивающих рамок. Используется рамочная разметка общей части лабораторной работы №5. Для вариантов 5–7 дополнительная полигональная разметка из индивидуальной части сохраняется отдельно: её нельзя подставлять вместо рамочной разметки в приведённые ниже программы.

Обучение нейронной сети

Нейронная сеть преобразует входное изображение в признаки, по которым определяются положение и класс объекта. Во время обучения предсказанные рамки и классы сравниваются с разметкой. Функция потерь численно характеризует расхождение, а алгоритм оптимизации изменяет веса сети так, чтобы уменьшить это расхождение на обучающих примерах.

Эпохой называется один проход по обучающей выборке. Изображения поступают в сеть группами — пакетами. Размер пакета задаётся параметром batch и влияет на требуемую память и характер обновления весов. Скорость обучения lr0 определяет начальный масштаб этих обновлений. Уменьшение обучающей ошибки само по себе не подтверждает, что модель правильно обрабатывает новые изображения.

В данной работе используется YOLOv8, поскольку набор из лабораторной работы №5 уже подготовлен для этой модели. Основной вариант — YOLOv8n. Буква n обозначает малую конфигурацию модели; в одном из индивидуальных заданий она сравнивается с YOLOv8s. Конкретную конфигурацию выбирают по результатам опыта, а не только по размеру файла весов [1].

Предварительно обученная модель загружается из файла yolov8n.pt. При обучении на собственных данных её веса адаптируются к выбранным классам. Загрузка yolov8n.yaml создаёт модель с той же архитектурой без переноса предварительно обученных весов. Это различие исследуется в варианте 10. На небольшом наборе обучение с нуля может не дать пригодного результата за отведённое число эпох.

Разделение данных и согласование предобработки

Обучающая выборка train используется для изменения весов. Валидационная выборка valid используется для сравнения режимов обучения. Тестовая выборка test сохраняется для итоговой проверки в лабораторной работе №7. В файле data.yaml путь к валидационной выборке задаётся ключом val, хотя сам каталог после экспорта из Roboflow обычно называется valid.

Состав выборок и порядок классов должны совпадать с окончательной версией лабораторной работы №5. Кадры одного ролика или одной непрерывной серии съёмки не следует распределять между разными выборками. Если в предыдущей работе использовалось случайное разбиение соседних кадров, до начала обучения необходимо сформировать версию с разделением по роликам или независимым сериям. Новый состав фиксируется один раз для обоих опытов.

Минимум 40 изображений из лабораторной работы №5 достаточен для освоения порядка действий, но не подтверждает качество модели для реального производства. Проверьте число рамок каждого класса в train, valid и test. Если класс отсутствует в valid или test, качество его обнаружения на соответствующей выборке оценить нельзя. Набор дополняют до обучения, сохраняя независимость серий съёмки.

Сохраните последовательность обработки из лабораторной работы №3. Если разметка выполнена на изображениях после коррекции яркости или фильтрации, при дальнейшем применении модели требуется та же обработка. Если рамки поставлены на изображениях 640 × 640 пикселей с дополнением полей, проверочные кадры необходимо подготовить тем же способом. Изменение размера внутри YOLO не заменяет ранее выполненную коррекцию яркости.

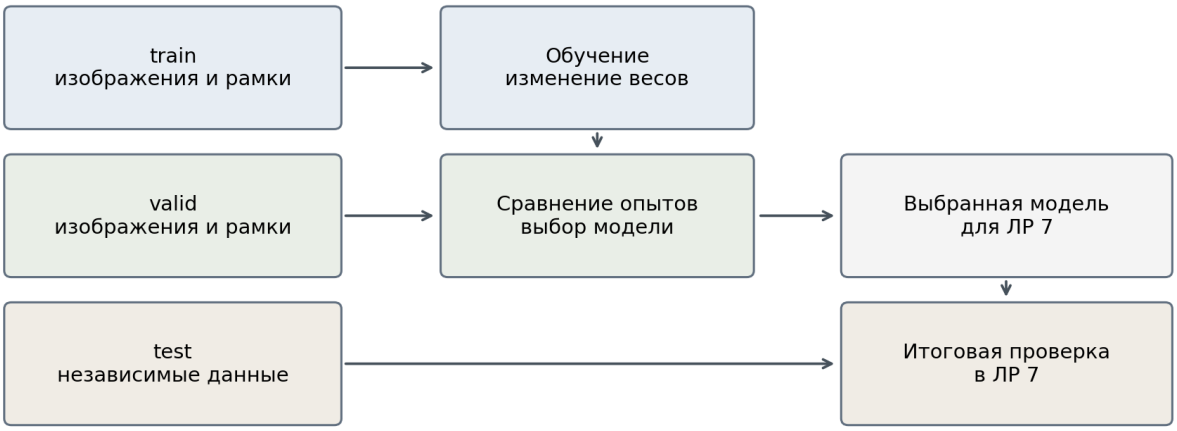


Рисунок 1 — Использование выборок при обучении и итоговой проверке модели

Изменение обучающих изображений

Искусственное изменение обучающих примеров называется аугментацией. Оно может включать поворот, изменение масштаба, отражение и изменение яркости. Такие преобразования допустимы, если они соответствуют возможным условиям наблюдения. Например, отражение надписи может создать изображение, которое не встречается при реальной съёмке.

В опытах ниже дополнительные преобразования исходно отключены. Для вариантов 5–8 включается только преобразование, указанное в таблице заданий. Геометрические преобразования изображений и рамок выполняются средствами библиотеки совместно. Валидационные и тестовые изображения искусственно не изменяются при выборе режима обучения [2].

Оценка результата обучения

Правильность локализации характеризуется отношением площади пересечения предсказанной и эталонной рамок к площади их объединения. Этот показатель обозначается IoU. Совпадение класса и достаточное перекрытие необходимы для зачёта обнаружения. Порядок сопоставления рамок подробно рассматривается в лабораторной работе №7.

Показатель AP характеризует соотношение точности и полноты обнаружения одного класса при изменении порога уверенности. Показатель mAP представляет собой среднее AP по оцениваемым классам. В mAP@0.5 используется порог перекрытия 0,5, а mAP@0.5:0.95 усредняет результат по десяти порогам от 0,5 до 0,95 с шагом 0,05. Второй показатель строже оценивает точность положения рамок [3].

Для выбора из двух опытов используется mAP@0.5:0.95 на valid. Дополнительно фиксируются mAP@0.5, продолжительность обучения и размер файла весов. Если разница mAP@0.5:0.95 не превышает 0,01, в работе допускается предпочесть менее затратную

модель, объяснив выбор. Это учебное правило сравнения, а не статистический критерий равенства моделей.

Алгоритм сравнения режимов обучения

Сначала фиксируются версия набора, состав выборок и общие параметры. Затем формируются два опыта A и B, различающиеся одним заданным фактором. Для каждого опыта модель загружается заново. После обучения оценивается сохранённый файл best.pt на одной и той же valid. Результаты записываются в общую таблицу, по которой выбирается модель для лабораторной работы №7.

В обоих опытах устанавливается seed = 42 и используется один вычислитель. Фиксация начального значения генераторов уменьшает влияние случайности, однако не гарантирует полного совпадения результатов на разных устройствах и версиях программ. При малой разнице показателей вывод следует формулировать осторожно.

Порядок выполнения работы

Работа состоит из общей части и индивидуальной части. Номер варианта и производственный объект сохраняются из лабораторной работы №5. Все программы запускаются из корневого каталога project. Команды установки выполняются в терминале выбранной среды Python.

Общая часть

Шаг 1 Подготовка программной среды

Для работы используются Python, Ultralytics, PyTorch, OpenCV, PyYAML и NumPy. Допускается использовать локальный компьютер или среду Google Colab. Наличие графического ускорителя сокращает время обучения; при его отсутствии программа выбирает центральный процессор. До опыта проверьте, что установленная сборка PyTorch видит доступное устройство.

python -m pip install ultralytics opencv-python pyyaml numpy

python -m pip freeze > requirements.txt

Сохраните версию Python, перечень пакетов, наименование процессора, объём оперативной памяти и, при наличии, графического ускорителя. Файлы yolov8n.pt и yolov8s.pt могут загружаться при первом обращении. При отсутствии доступа к сети их предварительно размещают в каталоге проекта. Полученные веса best.pt из этой работы сохраняют отдельно от исходных весов.

Шаг 2 Подготовка структуры проекта

Скопируйте окончательный экспорт рамочной разметки из лабораторной работы №5 в каталог dataset. Каталоги изображений и аннотаций должны иметь следующую структуру.

Каждому изображению соответствует файл .txt с тем же основным именем. Для проверенных кадров без объектов создаются пустые файлы аннотаций.

```
project/
  dataset/
    data.yaml
    train/images/    train/labels/
    valid/images/    valid/labels/
    test/images/     test/labels/
  prepare_dataset.py
  train_compare.py
  preprocess.py
  requirements.txt
  runs/lr6/          # создаётся при обучении
  results/           # таблицы и описание выбранной модели
```

Запишите число изображений и рамок каждого класса в каждой выборке. Убедитесь, что одинаковые изображения и кадры одной серии не оказались в разных выборках. Для анализа можно использовать алгоритмы лабораторной работы №5. При обнаружении ошибок исправьте набор до запуска обоих опытов.

Шаг 3 Проверка описания набора

В экспорте Roboflow могут использоваться относительные пути с переходом к родительскому каталогу. Программа ниже рассчитана на структуру шага 2: она сохраняет перечень классов, проверяет наличие каталогов и явно задаёт корневой путь набора. Исходное описание сохраняется как data_original.yaml. Создайте файл prepare_dataset.py.

```
from pathlib import Path
import shutil
import yaml

root = Path("dataset").resolve()
path = root / "data.yaml"
cfg = yaml.safe_load(path.read_text(encoding="utf-8"))
names = cfg["names"]
if isinstance(names, dict):
    names = {int(k): v for k, v in names.items()}
    if sorted(names) != list(range(len(names))):
        raise ValueError("Номера классов должны идти от 0 подряд")
```

```
else:
    names = dict(enumerate(names))
if not names or len(set(names.values())) != len(names):
    raise ValueError("Проверьте перечень классов")
for split in ("train", "valid", "test"):
    for subdir in ("images", "labels"):
        if not (root / split / subdir).is_dir():
            raise FileNotFoundError(root / split / subdir)
backup = root / "data_original.yaml"
if not backup.exists():
    shutil.copy2(path, backup)
cfg = dict(path=str(root), train="train/images",
           val="valid/images", test="test/images", names=names)
path.write_text(yaml.safe_dump(cfg, allow_unicode=True),
                encoding="utf-8")
print(path)
print(names)
python prepare_dataset.py
```

Проверьте, что номер каждого класса в аннотациях соответствует его имени в data.yaml. Приведённая программа проверяет описание и каталоги; содержимое разметки проверяется средствами лабораторной работы №5. Наличие файла data.yaml само по себе не означает, что разметка правильная.

Шаг 4 Сохранение последовательности предобработки

Создайте preprocess.py и перенесите в функцию prepare_frame выбранную последовательность операций из лабораторной работы №3 с теми же параметрами и порядком. На выходе должно получаться цветное изображение BGR типа uint8, соответствующее изображениям, на которых выполнена разметка. Если внешняя обработка не применялась, оставьте приведённый вариант без изменений.

```
def prepare_frame(frame):
    prepared = frame.copy()
    # Здесь размещаются выбранные операции из ЛР 3.
    return prepared
```

Повторно обрабатывать уже подготовленные изображения dataset не нужно. Функция предназначена для новых кадров. Для проверки сопоставьте результат функции

на одном исходном кадре с его изображением из размеченного набора. При несовпадении разрешения, яркости или полей устраните причину до дальнейшей работы.

Шаг 5 Задание условий двух опытов

По таблице вариантов определите изменяемый фактор и два его значения. Остальные параметры одинаковы. В программе ниже номер варианта задаётся переменной VARIANT. Размер входа imgsz может отличаться от размера хранящегося изображения: библиотека сама подготавливает изображение и рамки для обучения.

Общие условия: YOLOv8n с предварительными весами, 40 эпох, imgsz = 640, batch = 4, оптимизатор AdamW, lr0 = 0,001. Параметр patience = 0 отключает досрочное завершение, чтобы заданное число эпох было одинаково воспроизводимо в пределах выбранного режима. Для сравнительного опыта автоматический выбор оптимизатора не используется [2].

В вариантах 1–2 меняется размер входа, в варианте 3 — число эпох, в варианте 4 — число начальных модулей, для которых запрещено обновление весов. В вариантах 5–8 изменяется один вид аугментации. В варианте 9 сравниваются две конфигурации сети, в варианте 10 — два способа начальной установки весов. Замораживание модулей в варианте 4 необходимо проверить по сообщениям библиотеки.

Шаг 6 Обучение и оценка на valid

Создайте train_compare.py. Программа выполняет оба опыта последовательно и сохраняет измеренные результаты. Код предназначен для запуска как отдельный файл, в том числе в Windows. Каждый опыт начинается с новой загрузки модели и начальных установок генераторов случайных чисел.

```
from pathlib import Path
import csv
import json
import random
import time
import numpy as np
import torch
from ultralytics import YOLO

VARIANT = 1                    # свой номер варианта
CASES = {
    1: ("imgsz", 416, 640),
    2: ("imgsz", 640, 960),
```

```
    3: ("epochs", 20, 40),
    4: ("freeze", 0, 10),
    5: ("hsv_v", 0.0, 0.3),
    6: ("fliplr", 0.0, 0.5),
    7: ("degrees", 0.0, 10.0),
    8: ("scale", 0.0, 0.3),
    9: ("model", "yolov8n.pt", "yolov8s.pt"),
    10: ("model", "yolov8n.pt", "yolov8n.yaml"),
}

def main():
    device = 0 if torch.cuda.is_available() else "cpu"
    root = Path.cwd()
    project = root / "runs" / "lr6"
    output = root / "results"
    output.mkdir(exist_ok=True)
    data = root / "dataset" / "data.yaml"
    common = dict(
        data=str(data), epochs=40, imgsz=640, batch=4,
        device=device, workers=0, seed=42,
        deterministic=True, amp=False, patience=0,
        optimizer="AdamW", lr0=0.001, lrf=0.01,
        freeze=0, mosaic=0.0, mixup=0.0,
        hsv_h=0.0, hsv_s=0.0, hsv_v=0.0,
        degrees=0.0, translate=0.0, scale=0.0,
        shear=0.0, perspective=0.0,
        flipud=0.0, fliplr=0.0, close_mosaic=0,
        plots=True, save=True, exist_ok=False,
    )
    factor, value_a, value_b = CASES[VARIANT]
    rows = []
    for tag, value in (("a", value_a), ("b", value_b)):
        name = f"v{VARIANT:02d}_{tag}"
        if (project / name).exists():
            raise FileExistsError(f"Каталог уже существует: {name}")
```

```
random.seed(42)
np.random.seed(42)
torch.manual_seed(42)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(42)
cfg = common.copy()
source = "yolov8n.pt"
if factor == "model":
    source = value
else:
    cfg[factor] = value
model = YOLO(source)
started = time.perf_counter()
model.train(project=str(project), name=name, **cfg)
train_s = time.perf_counter() - started
run = Path(model.trainer.save_dir)
weights = run / "weights" / "best.pt"
best = YOLO(str(weights))
metrics = best.val(
    data=str(data), split="val", imgsz=cfg["imgsz"],
    batch=1, device=device, workers=0,
    conf=0.001, iou=0.7, plots=True,
    project=str(project), name=name + "_valid",
)
row = dict(
    run=name, factor=factor, value=value,
    source=source, imgsz=cfg["imgsz"],
    epochs=cfg["epochs"], train_s=round(train_s, 2),
    map50=float(metrics.box.map50),
    map50_95=float(metrics.box.map),
    weights_mb=round(weights.stat().st_size / 1e6, 3),
    weights=str(weights),
)
rows.append(row)
(output / (name + ".json")).write_text(
```

```
            json.dumps(row, ensure_ascii=False, indent=2),
            encoding="utf-8")
    with (output / "lr6_compare.csv").open(
            "w", newline="", encoding="utf-8-sig") as file:
        writer = csv.DictWriter(file, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)
    print(rows)


if __name__ == "__main__":
    main()
python train_compare.py
```

Параметр iou = 0,7 в вызове val управляет подавлением перекрывающихся предсказаний. Он не заменяет пороги IoU, по которым рассчитываются mAP@0.5 и mAP@0.5:0.95. Значение conf = 0,001 оставляет низкие по уверенности предсказания для построения зависимости точности от полноты; рабочий порог для видео будет выбран отдельно в лабораторной работе №7 [3].

Если памяти недостаточно, уменьшите batch для обоих опытов и начните их заново. При изменении числа эпох или размера входа из-за ограничений вычислителя зафиксируйте согласованные значения в отчёте. Не сравнивайте опыт, завершённый по ошибке, с полностью выполненным обучением. Для повторного запуска используйте новый каталог project, сохранив результаты предыдущего опыта.

Индивидуальная часть

Шаг 7 Анализ изменяемого фактора

Объясните, какое свойство обучения изменяется по вашему варианту. Для параметров аугментации приведите примеры допустимых преобразований выбранного производственного объекта. Для вариантов 1–2 проверьте, сохраняются ли мелкие признаки при уменьшении входа; для варианта 4 укажите, какие модули были заморожены; для вариантов 9–10 поясните различие сравниваемых моделей.

Сопоставьте кривые из results.png и значения в results.csv для обоих опытов. Найдите участок, на котором обучающая ошибка продолжает снижаться, а валидационное качество перестаёт расти или ухудшается. Если такого участка нет, не делайте вывод о переобучении только по разнице ошибок train и valid. Файл best.pt содержит модель, выбранную библиотекой по валидационному критерию, и может отличаться от last.pt последней эпохи.

Шаг 8 Сравнение моделей

Заполните таблицу фактически полученными значениями. Продолжительность обучения приводится для одного и того же устройства; размер файла характеризует хранение весов и не равен потреблению оперативной памяти. Не подставляйте ожидаемые показатели вместо измерений.

Опыт	**Изменяемый фактор и значение**	**Эпохи**	**Размер входа**	**mAP@0.5**	**mAP@0.5:0.95**	**Время, с**	**Веса, МБ**
A	По варианту	—	—	—	—	—	—
B	По варианту	—	—	—	—	—	—

Просмотрите не менее пяти валидационных изображений с предсказаниями. По возможности включите верное обнаружение, пропуск, лишнюю рамку и ошибку класса. Если какого-либо вида ошибки нет на valid, укажите это. Объясните, как найденные ошибки связаны с признаками, исследованными в лабораторной работе №4, и разметкой из лабораторной работы №5.

Шаг 9 Сохранение выбранной модели

Выберите опыт по установленному в работе правилу и обоснуйте решение. Сохраните путь к best.pt, значение imgsz, порядок классов, версию набора, состав выборок, параметры предобработки, версию программной среды и полную конфигурацию из args.yaml каталога обучения. Для лабораторной работы №7 создайте results/selected_model.json по образцу ниже. Путь к весам задаётся относительно корня project; значения измените по своему результату.

```
{
  "weights": "runs/lr6/v01_b/weights/best.pt",
  "imgsz": 640,
  "dataset_version": "Версия из ЛР 5",
  "preprocessing": "Описание функции prepare_frame"
}
```

Тестовую выборку на данном этапе не используйте. Модель считается подготовленной для следующей работы, если файл весов загружается, перечень классов соответствует data.yaml, результаты обоих опытов сохранены и выбор объяснён по valid. Универсальное минимальное значение mAP для всех производственных задач в этой работе не задаётся.

Содержание отчёта

1. Цель работы, номер варианта, производственный объект и классы.

2. Источник и версия набора из лабораторной работы №5, число изображений и рамок по классам в train, valid и test.

3. Состав программной среды и характеристики вычислителя.

4. Файл data.yaml и описание согласования предобработки.

5. Обоснование конфигурации сети и изменяемого фактора.

6. Параметры опытов A и B, листинг программы обучения.

7. Кривые обучения и таблица измеренных результатов на valid.

8. Не менее пяти валидационных кадров с разбором предсказаний.

9. Обоснование выбора модели и перечень файлов для лабораторной работы №7.

10. Вывод о влиянии изученного фактора и ограничениях полученного результата.

Варианты заданий

Общая часть выполняется всеми студентами. Индивидуальная часть включает сравнение двух значений одного фактора. Производственные объекты и классы продолжают таблицу лабораторной работы №5. Если ранее названия классов были согласованно изменены, сохраняется фактический порядок из data.yaml.

№	Производственный объект и классы	Фактор опыта A	Фактор опыта B
1	Детали на конвейере — gear, washer, nut	imgsz = 416	imgsz = 640
2	Печатная плата — resistor, capacitor, chip	imgsz = 640	imgsz = 960
3	Упаковка и тара — box, bottle, can	epochs = 20	epochs = 40
4	Маркировка — label, barcode	freeze = 0	freeze = 10
5	Металлическая поверхность — scratch, dent, rust	hsv_v = 0	hsv_v = 0,3
6	Пищевой продукт — apple, orange, defect	fliplr = 0	fliplr = 0,5
7	Сварной шов — weld, defect	degrees = 0	degrees = 10
8	Текстильное полотно — knot, hole, stain	scale = 0	scale = 0,3
9	Инструмент — wrench, hammer, pliers	YOLOv8n с весами .pt	YOLOv8s с весами .pt
10	Автомобильная деталь — disc, headlight, mirror	YOLOv8n с весами .pt	YOLOv8n из .yaml

Контрольные вопросы

1. Какие результаты лабораторных работ №3–5 необходимы для обучения модели?

2. Чем обучение нейронной сети отличается от применения готовой модели?

3. Что изменяется при использовании предварительно обученных весов на новом наборе классов?

4. Что означают эпоха, размер пакета и начальная скорость обучения?

5. Почему в данной работе рабочий порог уверенности не выбирают по test?

6. Как соседние кадры одного ролика в разных выборках влияют на оценку качества?

7. Почему требуется одинаковая предобработка при подготовке набора и обработке нового видео?

8. Что характеризуют mAP@0.5 и mAP@0.5:0.95?

9. Какой параметр изменён в вашем варианте и какое действие он выполняет?

10. Почему уменьшение обучающей ошибки не всегда сопровождается улучшением valid?

11. Чем различаются файлы best.pt и last.pt?

12. Какие сведения, кроме файла весов, необходимо передать для воспроизводимой проверки модели?